

Library Management using Microservices with Spring Boot

Problem Statement

Background:

A popular library ("Amazing Books") in the city is planning to launch a portal for expanding their reach. They hire you to help them, go online. They want to proceed with incremental but fast-paced releases to attract more readers. They have a few major functions which should be independently developed, tested, and released from other functions to enable fast-paced releases. Apart from this, they expect the user base to grow soon and thus must be highly scalable without compromising on availability, resiliency, and security. The portal would need to support different types of devices eg. web, mobile, tablet, and thus would require RESTful APIs which you need to build to manage issuing of books to the readers. Development of GUI (Graphical User Interface) would be taken up by another team and the delivery of the books will be handled by another vendor.

So, there are three major functions that need to be implemented:

1. APIs to fetch/add/edit/delete books (H2 Database).
2. APIs to fetch/add/cancel issuing of the books to the customers (H2 Database).
3. Customers to be managed by OAuth2 server(in-memory)

Apart from these functional requirements, you would also have to take care of non-functional requirements:

1. It should be easy to scale.
2. It must be highly secured.
3. It should be resilient.
4. It should be performant.
5. It should support quick releases.

Considering the above-mentioned requirements (functional + non-functional), Microservices Architecture seems fit for this. Before the implementation do provide High Level Design.

Below is a list of important functions(microservices) and the model which you may consider for the implementation:

1. Book(bookms): fetch/add/edit/delete books

Model: isbn, title, publishedDate, totalCopies, issuedCopies, author

Assumption: A book will have only one author

2. Issue books to customers(issuersms) - Fetch(REST call) Book Details eg bookId and availableCopies(totalCopies - issuedCopies) - Issue the book to the customer, if the book is available - Update issuedCopies in bookms - To keep things simple, these operations may not necessarily be atomic

Model: isbn, custId, noOfCopies

Note: A book(isbn) can be assigned to more than one customer

4. A service for dynamic service discovery

5. A service for handling cross-cutting concerns 5. A service for managing security - Have one client application registered(in-memory) - Add customers as in-memory user